PROCESS-IMPROVEMENT GOVERNANCE DOSSIER
Task Acceptance and Privileged Review Governance Dossier
Implementation completion, independent work-product acceptance, and Feature aggregate closure
Date: 17 August 2026
Repository: autodocs
Task: 0039-04
Status: governance dossier; not an assessment, product approval, or reviewer assignment
Contents

Date: 17 August 2026 Repository: autodocs Related work: reserved Feature 0039, Task 0039-04 Companion normative process: docs/pipeline/task-acceptance.md Status: governance dossier and implementation rationale; not an Automotive SPICE assessment, product approval, or reviewer assignment
Authority boundary. This dossier does not grant privilege, accept a Task or Feature, approve architecture, authorize release or external mutation, accept safety/security/privacy risk, or establish an Automotive SPICE capability level.
Executive summary
The repository historically used [x] and [w] as terminal Task markers. A committed deliverable or disposition could satisfy dependencies and contribute to Feature closure. Retained evidence, especially the Feature 0021 archive, demonstrates why that is insufficient: committed work, focused green checks, and apparently complete child markers can coexist with missing end-to-end behavior, weak or stale evidence, unrealistic validation, unresolved authority, or failure to achieve the Feature outcome.
The convention separates three decisions:
1 Implementation or disposition completion. The owner produces and commits the result, required validation, findings disposition, and real REF. The Task becomes [x] or [w].
2 Task work-product acceptance. A separately assigned, currently privileged, normally independent reviewer deeply inspects the exact Task baseline and the transitive closure of non-accepted prerequisites. A successful decision adds Acceptance: ✓ while retaining the [x] or [w] disposition.
3 Feature aggregate acceptance. After every active Task and Subtask has a current accepted disposition, a separate review checks Feature-level goal achievement, integration, negative and recovery behavior, findings, risks, prerequisites, and an exact aggregate manifest. Only then may the Feature move to DONE.md.
Acceptance is orthogonal to the checkbox marker. This preserves the difference between completed implementation and accepted wontfix, avoids the non-standard Markdown checkbox [✓], and permits staged migration of legacy parsers. Ordinary successors may start from [x] or [w] to preserve throughput, but acceptance procee

ds bottom-up and cannot cross an unaccepted prerequisite. High-risk contracts ca n explicitly require acceptance before successor implementation starts.
Grunts can implement, validate, commit, and prepare review packages. They cannot create, edit, invalidate, or remove acceptance, claim reviewer status, request generic runner acceptance, or move Features to DONE.md. Privilege is necessary b ut not sufficient: the reviewer also needs exact current assignment, competence, independence, and appropriate scope. Work-product acceptance never substitutes for architecture, product, release, safety, security, privacy, legal, signing, o r external-service authority.
The human process and instruction boundary are introduced by Task 0039-04. Machi ne enforcement cannot be improvised because current legacy doctors, editors, tra nsaction coordinators, automation policy, Feature 0037 lifecycle schemas, migrat ion contracts, and generated views encode [x] and [w] as terminal. Reserved foll ow-up Task 0039-05 owns coordinated enforcement and migration after the active t ransaction/editor/result prerequisites finish.
Key design choices
Choice
Decision
Rationale
Accepted representation
Keep [x] or [w]; add an exact structured Acceptance: ✓ record
Retains the actual disposition and avoids breaking Markdown checkbox parsing
Implementation dependency
[x] or [w] normally satisfies a successor start gate
Avoids serializing all grunt work behind privileged review
Acceptance dependency
All required predecessors must have current accepted dispositions
Prevents a Task from laundering unreviewed prerequisites into acceptance
High-risk dependency
Task contract may require accepted predecessor before implementation
Reduces downstream rework for canonical, irreversible, security, credential, arc hitecture, or release boundaries
Reviewer
Currently privileged, explicitly assigned, competent, normally independent
Privilege alone is not approval authority or independence
Self-review
Prohibited by default; explicit bounded authority waiver only
Prevents privileged implementation from becoming automatic self-approval
Feature closure
All Tasks/Subtasks accepted plus separate aggregate review
Child acceptance is necessary but does not prove integration or Feature outcome
Historical DONE
Preserve prior evidence status; do not retroactively claim acceptance
Avoids fabricated evidence and unbounded retrospective relabeling
Tool migration
Extend existing editor/transaction path; no second writer
Prevents competing authority and partial lifecycle semantics
Documents and processes that must change
Changes introduced with Task 0039-04
Path or process
Adaptation
Why it is needed
TODO.md header
Defines [x] and [w] as awaiting acceptance, orthogonal ✓, two dependency gates, reviewer authority, and Feature closure
It is the current authoritative marker and prerequisite contract
DONE.md header
Grandfathers historical entries and requires Task/Subtask acceptance plus aggreg ate review for future moves
Prevents retroactive claims and checkbox-only Feature closure
AGENTS.md
Separates implementation claim completion from acceptance; prohibits grunt promo

tion; defines assigned privileged review and aggregate closure
It governs daily coordination and bookkeeping
SANDBOX.md
Places acceptance and Feature movement outside grunt and generic runner authorit
y
It governs capability and execution boundaries
PRIVILEGED.md
Defines assignment, independence, inspection, validation, decision, evidence, an
d invalidation rules
Direct execution capability must not become self-approval
docs/pipeline/task-acceptance.md
Provides the normative review and closure process
Reviewers and implementers need one operational source
docs/pipeline/README.md
Catalogs the new process
Agents must be able to discover it
Feature 0039
Adds 0039-04, makes 0039-01 consume the convention, and adds reserved enforcemen
t Task 0039-05
Keeps implementation and deferred migration traceable
Coordinated machine-enforcement migration under Task 0039-05
The following artifacts encode the old terminal model and must change together.
Task 0039-04 does not appropriate active owners or introduce a competing lifecyc
le writer.
Area and exact paths
Current assumption
Required adaptation
_src/tools/legacy_task_doctor.py, tests, docs/pipeline/legacy-task-doctor.md
TERMINAL_MARKERS is {x,w}; terminal children make a Feature closure-eligible
Model implementation completion separately from current acceptance; validate rev
iew refs, prerequisite acceptance, invalidation, Feature aggregate eligibility,
and authority
_src/tools/legacy_task_editor.py, tests, fixtures, docs/pipeline/legacy-task-edi
tor.md
Closed operation set ends at [x]/[w]; parent aggregation consumes terminal marke
rs
Retain implementation operations; add a separately authorized acceptance candida
te/action and invalidation history without a second parser/writer
_src/tools/runner_transaction.py, tests, docs/pipeline/runner-transaction.md
Closure renderer changes [p] to [x] and treats bookkeeping as final
Keep grunt transaction limited to implementation completion; reject acceptance/F
eature move; later support only an approved typed privileged acceptance action
_src/tools/automation_safety.py, policy/tests, docs/pipeline/automation-safety.m
d
Dispositions expire when owner/expiry Task becomes x/w
Distinguish implementation and accepted expiry policies; detect unauthorized acc
eptance edits and Feature movement
issues/_schema/issue-item-v1.schema.json, issue-closure-v1.schema.json, fixtures
 and validators
Canonical closed state and immutable closure.json conflate completion and accept
ance
Add versioned acceptance assignment/decision/current-state/invalidation objects
while preserving disposition separately
docs/pipeline/issue-lifecycle.md, issue-store.md, issue-yaml-profile.md
Legacy [x]/[w] maps directly to closed terminal issue items
Reconcile closure versus acceptance, add paths/records, role matrix and current-
state derivation from append-only events
docs/pipeline/issue-migration-shadow-import.md, issue-cutover-rollback.md, schem
as and fixtures
Marker migration directly creates future closed state
Import completion and acceptance independently, preserve pre-policy history, and
 block cutover on acceptance-model drift

agent-workflow.json, instruction bundles, agent-workflow.md, agent-execution.md
Selector/actions do not identify acceptance authority or review action
Add policy/epoch-bound reviewer capability, assignment, stale-client rejection a
nd no-grunt action registry
Feature 0037 architecture package, approval, queue actions, graph/public project
ion
Review-ready package was assembled from old closure semantics
Refresh package digests/findings before approval; represent public acceptance wi
thout leaking internal identities/evidence
_src/validate.py and profile tests
Project validation can pass while tools still treat x/w as Feature-terminal
Add unauthorized-transition, evidence, prerequisite closure, invalidation, aggre
gate gate, and old-tool fail-closed checks
Existing evidence mechanisms should be reused: task-validation reports, environm
ent fingerprints, content-addressed artifact manifests, provenance relations, fi
ndings, collision planning, and path-isolated commits. Acceptance must not creat
e a second evidence store, issue queue, runner protocol, or closure editor.
Daily operating implications
For grunts and implementation owners
The implementation claim ends at [x] or [w]. The owner commits the substantive r
esult, implementer validation, findings disposition and real REF; creates a revi
ew-ready acceptance package; finalizes the claim; and moves to another implement
ation-ready Task. Waiting for review does not retain write ownership and is not
[u].
The acceptance package becomes part of normal Definition of Done. It should incl
ude the exact Task contract and digest, commits/tree, work-product manifest, dir
ect and derived scope, criterion-to-evidence matrix, prerequisite graph, immutab
le validation results, findings, authority/risk interfaces, migration/recovery b
ehavior and provenance. This is extra effort, but it reduces the much larger cos
t of reviewers reconstructing context from conversations and mutable logs.
A rejected review returns actionable corrections to [p] under a normal claim. An
 inconclusive review usually leaves [x] or [w] awaiting corrected evidence. Grun
ts do not alter the review decision or acceptance history.
For privileged reviewers
Acceptance work is a distinct queue and does not start autonomously. The current
 user or registered authority assigns the exact Task, prerequisite-closure batch
, or Feature baseline. The reviewer checks privilege, assignment, independence,
competence, authority epoch and evidence access before review.
The reviewer expands the transitive prerequisite graph and stops only at current
, reachable, non-invalidated acceptance records. Every other prerequisite is rev
iewed in topological order. The reviewer inspects the actual contract, meaningfu
l work-product changes, scopes, findings, negative/failure/recovery paths and sp
ecialist authority records. They independently evaluate or rerun focused, policy
, canary, negative and risk-based broader validation. They record exactly accept
ed, rejected or inconclusive.
Review evidence is committed before a separate path-isolated bookkeeping commit
adds acceptance credit with the real review REF. A relevant baseline change crea
tes additive invalidation and impact analysis; it never erases prior history.
For Feature owners and scheduling
Feature planning gains a visible acceptance queue. Teams should monitor implemen
tation-to-acceptance lead time, queue age, prerequisite-closure size, reviewer c
ompetence needs and Features waiting on aggregate review. Related prerequisite-c
losed reviews may be batched to reuse context, but every Task keeps an individua
l decision.
Reviewer capacity becomes a managed resource. Lack of capacity must not create s
elf-acceptance, false human blocks, or implementation claims held open indefinit
ely. High-risk interfaces should receive acceptance earlier to reduce downstream
 rework; low-risk implementation chains can be reviewed in coherent batches.
Benefits
1 Produced is separated from trusted. A runner success or implemen
ter assertion no longer implies independent acceptance.
2 Grunt delegation becomes safer. Grunts retain broad productive s

cope without authority to approve their own results.
3 Prerequisite assurance improves. A consumer cannot become accepted while relying on an unaccepted or invalidated foundation.
4 Feature closure becomes outcome-oriented. Integration, goal achievement, end-to-end behavior and aggregate risk are reviewed instead of inferred from child checkboxes.
5 Evidence becomes reproducible. Exact contracts, commits, manifests, runs, dependencies and authority epochs make stale or mixed evidence visible.
6 Authority boundaries become clearer. Work-product acceptance is explicitly separate from product and specialist decisions.
7 Process learning improves. Rejections, inconclusive causes, rerun mismatches, invalidations, escapes and review load can be measured.
8 Historical overclaim is reduced. Existing DONE.md records retain their actual evidence status instead of being relabeled under a stronger later rule.

Risks and controls

Risk
Possible consequence
Control
Privileged-review bottleneck
Large acceptance debt and delayed Features
Allow implementation from [x]/[w]; plan capacity; batch coherent closure; measure queue age
Rubber-stamp acceptance
✓ becomes a cosmetic checkbox
Mandatory package, independent review, negative/canary checks, findings and review-quality audits
Recursive review explosion
A late Task pulls in a large unaccepted history
Accept foundational work incrementally; expose closure size; stop at valid digest boundaries
Downstream work on rejected inputs
Rework propagates
Acceptance-before-start for high-risk contracts; warn on long unaccepted chains
Privilege spoofing
Grunt/local actor adds acceptance
Authority registry/signatures, protected typed action, transition validation and audit history
Self-review bias
Defects survive acceptance
Independent assignment; conflict disclosure; explicit bounded waiver only
Acceptance mistaken for release/risk approval
Unauthorized operational decision
Role matrix and proper specialist decision references
Stale acceptance
Changed work remains trusted
Exact baseline binding, additive invalidation and dependency impact analysis
Evidence growth or privacy exposure
Repository bloat or restricted-data leak
Content-addressed compact manifests, classification/redaction, references instead of copies
Dual lifecycle writers
Old and new tools disagree
Extend existing editor/transaction path, one cutover epoch, fail closed on partial migration
Metric gaming
Fast approvals replace assurance
Measure escapes, invalidations, inconclusive causes and quality, not reviewer rankings or pass volume
Historical acceptance overclaim
Old [x] appears stronger than its evidence

Explicit grandfathering without retroactive acceptance
Rollout and migration
Phase 0 — policy and dossier
Task 0039-04 establishes the human policy, authoritative backlog semantics, agent boundaries, detailed review procedure, impact map and this dossier. It closes at [x] awaiting independent acceptance; the author does not bootstrap the convention by self-review.
Phase 1 — interim privileged review
Privileged review may use exact candidates and separate evidence/bookkeeping commits. Old automated “Feature closure eligible” diagnostics are advisory only. Any tool that cannot preserve acceptance records must fail closed. No grunt or generic runner path can add acceptance.
Phase 2 — coordinated machine enforcement
Reserved Task 0039-05 integrates acceptance with the existing structural editor and durable transaction/result path after Tasks 0038-02, 0038-05 and 0038-10. It adds authorization checks, assignment, decisions, invalidation, graph closure, Feature aggregate records, fixtures and unauthorized-transition rejection.
Phase 3 — Feature 0037 contract reconciliation
The review-ready issue-store lifecycle currently models closure as terminal. Before architecture approval or cutover, its schemas, migration, queue actions, authority policy, validators, package digests/findings and generated views must incorporate acceptance. The issue store remains shadow data until authorized cutover; no dual writable authority is introduced.
Historical and rollback policy
Existing DONE.md history remains evidence under prior semantics. It is neither automatically accepted nor automatically reopened. Open Features accumulate acceptance debt: their implementation-complete Tasks must be reviewed before Feature closure. Feature 0021 remains archived-not-accepted.
Acceptance-aware enforcement needs one explicit authority epoch. Partial migration must not permit both old checkbox-only Feature closure and new acceptance-aware closure. If acceptance validation is unavailable, Feature closure remains disabled while implementation can continue to [x] or [w].
Automotive SPICE relationship map
No assessment claim. Automotive SPICE capability is assessed for a named process and organizational/process-instance scope using an applicable licensed model and competent assessors. This convention can support evidence; it does not establish conformity, a process outcome, a process-attribute rating, or a capability level.
Area
How the convention can support it
What remains necessary
SUP.1 Quality Assurance
Independent conformance review, work-product/process checks, findings, escalation, correction verification and objective records
Organizational QA strategy, qualified assignment, sufficient independence, coverage, reporting and management resolution; privilege alone is not QA
SUP.2 Verification
Criteria-to-evidence traceability, planned review methods, independent reruns, negative tests, findings and results
Product/process-specific scope, methods, criteria, environment, competence and results; Task acceptance is not universal technical verification
SUP.8 Configuration Management
Exact commits, baselines, manifests, digests, status accounting, integrity checks, controlled promotion and invalidation
Full configuration strategy, item identification, access/release controls, audits, backup/restore and product-wide coverage
SUP.9 Problem Resolution Management
Stable findings, severity, owner, correction, closure evidence and recurrence/escape analysis
Correct classification and complete problem intake, analysis, resolution, communication and trend lifecycle
SUP.10 Change Request Management

Impact analysis, baseline invalidation, authorized disposition, implementation trace and reacceptance
Formal request intake, evaluation, approval/rejection/withdrawal, planning, implementation and closure; a Git diff is not automatically a change request
MAN.3 Project Management
Work packages, roles, dependencies, review queues, status, blockers and corrective work
Complete goals/scope, feasibility, estimates, schedule, resources, commitments, interfaces, communication and control
MAN.5 Risk Management
Risk-based review depth, specialist boundaries, escalation, residual-risk references and invalidation triggers
Risk identification, analysis, treatment, owner, monitoring, effectiveness and proper residual-risk authority
MAN.6 Measurement
Acceptance lead time, queue age, findings, rerun mismatch, invalidation and escape measures
Measurement objectives, definitions, data quality, collection, analysis, communication and demonstrated decision use
PIM.3 Process Improvement
Pilot, baseline, findings, measured evaluation, controlled rollout, tailoring and revision
Prioritized objectives, deployment, monitoring, effectiveness evaluation and sustained organizational adoption; the dossier alone is not deployment
REU.2 Reuse Program Management
Review packages, schemas, profiles and evidence patterns can become reusable process assets
Reuse strategy, asset classification, ownership, qualification, support and measurement
Capability-level implications
Level
Potential supporting evidence
What is insufficient
CL2 – managed
Acceptance objectives, roles, plans, monitoring, resources, controlled work products, findings and configuration evidence can support PA 2.1/2.2 considerations
One accepted Task, one checklist, or documentation without repeated managed execution
CL3 – established
A versioned standard process, competence model, templates, tailoring, infrastructure, deployment guidance, training and repeated use across applicable instances
Copying this dossier into instructions or one pilot; standard process definition and deployment must be demonstrated
CL4 – predictable
Stable measures could contribute to quantitative objectives, process-performance baselines/models, prediction and control after sufficient trustworthy data exists
Dashboards, counts, simple averages or target pass rates; statistical validity and process control are required, and exact attribute names depend on the selected model edition
CL5 – innovating/optimizing
Controlled experiments, quantitatively selected improvements, innovation deployment and sustained effectiveness could contribute
Frequent rule changes, more automation, AI use, suggestions, or a “continuous improvement” label without quantitative understanding and verified outcomes
The strongest immediate relationships are to quality assurance, verification, configuration/work-product management and controlled findings. Higher capability depends on broad, repeated, measured deployment—not on the sophistication of this convention.
Open policy decisions
Decision
Recommended default

Revisit when
Canonical acceptance storage after Feature 0037
Separate immutable acceptance object plus current-state projection; do not overl
oad closure.json
Final issue-store schema/event review
Authority registry and signatures
Explicit assignment plus policy-digested reviewer role and verified signed decis
ion for protected acceptance
Hosting/signing readiness and risk class
Independence threshold
No claim owner, principal implementer, decisive author, or sole validator; expli
cit waiver only
Small-team and specialist constraints
Acceptance-before-start profiles
Restrict to high-risk canonical, irreversible, credential/security, architecture
 and release boundaries
Observed downstream rework or bottleneck data
Review evidence and retention
Content-addressed internal manifest and immutable refs; privacy-safe public proj
ection
Privacy, storage and audit requirements
Sampling
Whole-population deterministic validation plus recorded risk-based sample; zero
tolerated material sample defect
Population scale, heterogeneity and validator capability
Acceptance service levels
Measure queue age first; set targets after a baseline
Capacity and delivery needs
Minor-finding deferral
Only when no criterion is defeated and owner/due/downstream Task are explicit
Risk-class policy
Open-Feature migration
Review implementation-complete Tasks before Feature closure; do not backfill his
torical DONE
Acceptance-debt volume and cutover profile
Emergency exceptions
Containment only, no acceptance, mandatory expiry and follow-up
Applicable incident/change authority
Operational checklists
Implementer handoff

- Exact Task/Feature contract and digest are identified.
- Substantive result and real REF are committed; unrelated work is
 excluded.
- Work-product and scope manifests include generated and external
effects.
- Every criterion maps to implementation, validation and evidence.
- Validation is fresh, immutable, canary-backed and covers negativ
e/recovery behavior.
- Findings have stable IDs, severity, owner, status and evidence.
- Prerequisite graph and current acceptance status are captured.
- Required specialist decisions are referenced from their proper a
uthorities.
- Task is [x] or [w]; implementation claim is finalized.
- No acceptance credit or Feature move was attempted.

Privileged Task review

- Exact current assignment, privilege, competence and independence
 are verified.
- Contract and candidate baseline are pinned with no material drif
t.
- Transitive non-accepted prerequisites are computed and topologic
ally ordered.
- Every criterion and meaningful work-product change is inspected.

- Direct, derived, external and evidence scope matches observed state.
- Validation coverage, freshness, canaries, findings and environment are evaluated.
- Focused, policy, negative/failure/recovery and risk-based broader checks are independently evaluated or rerun.
- Specialist authority records are verified without reviewer overr each.
- Critical and major findings are absent; minor deferrals are valid and traceable.
- One accepted, rejected or inconclusive decision is recorded append-only.
- Review evidence is committed before separate acceptance bookkeeping.

Feature aggregate review

- Every active Task/Subtask has a current accepted disposition.
- Prerequisite and conditional profile edges are complete and current.
- Feature goal and Feature Definition of Done are demonstrably met.
- Task outputs integrate on compatible exact baselines.
- Feature-level end-to-end, negative, migration and recovery evidence passes.
- Cross-Task findings, risks, exclusions and successors are dispositioned.
- Required product and specialist decisions are authentic and current.
- Aggregate manifest binds child acceptance and current work products.
- Independent aggregate review record is committed.
- Only then is the path-isolated DONE.md move authorized.

Part II – Normative privileged acceptance process

The following companion process is included in full so the DOCX/PDF dossier remains independently usable. The repository Markdown file docs/pipeline/task-acceptance.md remains the normative operational source.

Privileged Task Acceptance and Feature Closure

Status: Normative legacy-authority process introduced by reserved Task 0039-04. Until Feature 0037 completes its authorized cutover, TODO.md, DONE.md, and active coordination records remain authoritative. The future issue-store contracts must implement equivalent semantics before approval and cutover.

Purpose and boundary

Implementation completion and independent acceptance are different decisions. A Task may have a committed deliverable, successful validation, and a real REF while still resting on incomplete evidence, unrealistic tests, an unreviewed prerequisite, a hidden authority assumption, or a result that does not satisfy the intended outcome. This process introduces a separate Task-acceptance state, rendered as ✓, and an independent Feature aggregate-acceptance gate.

Task acceptance means that the exact reviewed work-product baseline satisfies the Task contract under the recorded review scope. It does not grant or imply product approval, architecture approval, release authorization, safety acceptance, cybersecurity/privacy residual-risk acceptance, external-service authorization, process-baseline approval, or an Automotive SPICE capability rating. The reviewer verifies that any separately required decision exists and is correctly bound; the reviewer does not manufacture that authority.

The word accepted in this document is namespaced to Task/Feature work-product acceptance. It is distinct from curation-item decisions, review-request acceptance, publication, or external approval processes elsewhere in docs/pipeline/.

State and rendering model

Acceptance is orthogonal to the legacy checkbox marker so that the executed disposition remains visible and current parsers are not silently broken.

Representation
Meaning

Ordinary implementation start gate
Feature closure
[ ], [?], [p], [u]
Existing open, investigatory, active, or genuinely human-blocked state
Unsatisfied
Unsatisfied
[x]
Implementation complete, committed, implementer validation complete, awaiting ac
ceptance
Satisfied unless the consumer explicitly requires prior acceptance
Unsatisfied
[w]
Non-implementation disposition complete, reason/evidence committed, awaiting acc
eptance
Satisfied unless the consumer explicitly requires prior acceptance
Unsatisfied
Acceptance: ✓
The exact [x]/[w] baseline has a current accepted disposition
Satisfied
Required, but not sufficient by itself
An accepted Task retains [x] or [w] on its header and adds a structured acceptan
ce record. This preserves the distinction between a delivered result and a corre
ctly accepted wontfix, superseded, duplicate, or cancelled disposition. [✓] is n
ot introduced as a Markdown checkbox because it is not a standard checkbox token
 and would erase the underlying disposition.
The minimum legacy rendering is:

```
  - **Acceptance:** ✓
    - **Disposition:** `completed|wontfix|superseded|duplicate|cancelled`
    - **Accepted by:** `<authorized-reviewer-identity>`
    - **Authority reference:** `<immutable assignment/authority reference>`
    - **Accepted at:** `<ISO-8601 timestamp with timezone>`
    - **Contract SHA-256:** `<64 lowercase hexadecimal>`
    - **Work-product manifest SHA-256:** `<64 lowercase hexadecimal>`
    - **Prerequisite-acceptance SHA-256:** `<64 lowercase hexadecimal>`
    - **Review REF:** `<full reachable 40-hex commit>`
```

A historical ARCHIVED — NOT ACCEPTED record never receives acceptance credit. Ex
isting Features already in DONE.md retain the semantics and evidence status reco
rded when they were moved; they are not retroactively relabeled or represented a
s accepted under this process.
Authority and separation of duties
Sandboxed/grunt agents may implement, investigate, validate, commit, prepare acc
eptance packages, and move their own claimed Tasks to [x] or [w]. They are prohi
bited from:

- creating, modifying, invalidating, or removing a current Accepta
nce: ✓ record;
- representing themselves as the acceptance reviewer;
- asking a generic runner action to perform acceptance promotion;
- moving a Feature to DONE.md;
- treating privilege, a green command, or a Task REF as acceptance
.

Only a session that is both currently privileged and explicitly assigned by the
current user or registered acceptance authority to the exact review scope may de
cide Task or Feature acceptance. Privilege alone is not acceptance authority. A
model name, Git author, claim filename, terminal access, or role self-assertion
is not proof.
The reviewer is independent by default: the reviewer must not be the Task claim
owner, principal implementer, author of the decisive technical disposition, or s
ole producer of the validation evidence. Prior consultation does not automatical
ly destroy independence, but material design authorship, implementation, or self
-generated approval evidence must be disclosed and normally disqualifies the rev
iewer. A self-acceptance exception requires an explicit current-user or register
ed-authority waiver naming scope, reason, duration, conflict, and compensating c

ontrols. Urgency or reviewer scarcity is not a waiver.
Specialist competence is part of assignment. One reviewer need not possess every specialist authority, but the review plan must identify required architecture, security, privacy, safety, release, legal, operational, or domain decisions and verify their authentic records.
Implementation completion and review handoff
The implementation owner completes the existing claim at [x] or [w], commits the substantive result and bookkeeping, finalizes the implementation claim, and returns to ordinary queue work. Waiting for acceptance must not hold the implementation write scope or become [u].
The acceptance package must identify:
1 Task and Feature identity, exact normative Task text, acceptance criteria, Definition of Done, and contract digest;
2 exact substantive and bookkeeping commits, candidate tree, expected parent/base, and authority epoch;
3 a complete authoritative work-product manifest with paths, roles, source/generated classification, media types, and digests;
4 declared and observed direct, derived, external, and evidence scopes, including proof that unrelated work was excluded;
5 a criterion matrix mapping every normative condition to implementation, validation, evidence, findings, and disposition;
6 the direct and transitive prerequisite graph plus existing acceptance records and invalidation state;
7 validation profiles, commands or typed actions, environment/input/output identities, coverage, canaries, negative cases, durations, resource bounds, and immutable results;
8 material findings, severity, affected criterion/artifact, owner, corrective action or authorized disposition, and verification status;
9 security, privacy, safety, external-effect, migration, compatibility, recovery, rollback, and residual-risk interfaces;
10 user-prompt/process provenance and immutable evidence references without secrets or restricted personal data;
11 prior rejected, inconclusive, superseded, or invalidated review attempts.
Missing, stale, mixed, inaccessible, malformed, or internally inconsistent package information yields inconclusive, never an assumed pass.
Privileged review procedure
1. Assignment and preflight
The reviewer verifies current privilege, exact user/authority assignment, independence, competence, review scope, policy/authority epoch, and absence of a competing review assignment. The reviewer pins the exact Task contract and candidate baseline before substantive inspection. Any drift requires a new review baseline.
2. Expand the prerequisite closure
The reviewer parses the exact prerequisite graph, rejects missing endpoints, self-edges, duplicate/reversed edges, cycles, ambiguous alternatives, and required prose-only dependencies, then computes the transitive prerequisite closure. A valid, reachable, non-invalidated acceptance record forms a review boundary. Every prerequisite without such a boundary enters the same review batch.
The batch is topologically ordered from leaves to the target. Acceptance is prerequisite-closed: a Task cannot be accepted while a required predecessor remains unaccepted, rejected, inconclusive, stale, or invalidated. Several items may be reviewed in one batch, but each receives its own decision and is promoted bottom-up.
Ordinary implementation may consume [x]/[w] to avoid serializing all work behind privileged reviews. A Task with an irreversible migration, canonical interface/schema, credential/security boundary, public release, architecture selection, or comparable high-risk dependency may state a stricter acceptance-before-start gate; until machine-enforced profile edges exist, this gate must be explicit in the Task contract and checked manually.
3. Inspect contract, work products, and scope
The reviewer reads the actual changed source, configuration, policy, process, te

st, and generated-output contracts—not only summaries or logs. Every criterion, changed authoritative path, authority-sensitive path, manifest/digest, structured finding, negative test requirement, and recovery/rollback boundary receives complete inspection.

For a large homogeneous generated population, deterministic whole-population checks remain preferred. Sampling is permitted only when the population is enumerated and digest-bound, a complete validator is infeasible or complementary inspection is useful, and the method records strata, seed, size, exclusions, boundary/high-risk selections, and rationale. Authority boundaries are never sampled. A material sample defect, population heterogeneity, missing canary, or unexplained mismatch expands the sample or triggers complete inspection.

The reviewer confirms that declared scope agrees with observed changes; no ambient staged, unstaged, untracked, generated, or external effect was silently included or omitted. Generated artifacts must map to their canonical producer and source manifest.

4. Evaluate and rerun validation

Existing immutable runs support review but do not replace independent freshness checks. Against an isolated exact candidate where feasible, the reviewer reruns:

- package/schema/digest/reachability and prerequisite checks;
- focused tests for changed behavior;
- required security/privacy/authority policy checks;
- representative negative, canary, failure, cancellation, retry, and recovery cases;
- broader regression, generation, or end-to-end validation required by the Task risk and contract.

A full expensive rerun may be omitted only when the validation profile permits reuse, all inputs/environment/tool versions and immutable results match exactly, canaries prove coverage, and the reviewer records why reproduction adds no material assurance. Child exit zero, output existence, timestamps, synthetic-only data, or a baseline-only run are not sufficient by themselves.

5. Review findings and authority boundaries

Findings use stable identities and at least critical, major, minor, and observation/improvement classes. Critical and major findings block acceptance. A minor finding may remain only when it does not contradict a criterion, the Task contract permits deferral, and it has an owner, due condition, traceable downstream item, and any required authority disposition.

The reviewer verifies required architecture, security, privacy, safety, release, external mutation, signing, or residual-risk decisions, but does not make them without the corresponding registered authority. An absent required decision blocks or makes the review inconclusive according to the evidence.

6. Decide and record

The review has exactly one outcome for the reviewed baseline:

- accepted: all criteria and prerequisite acceptance gates are satisfied; material findings are closed or validly dispositioned;
- rejected: evidence demonstrates a material nonconformity;
- inconclusive: identity, evidence, environment, scope, or authority is insufficient to determine conformity.

Rejected and inconclusive attempts remain append-only evidence. Rejection normally returns the Task to [p] when corrective implementation work is actionable. Inconclusive normally leaves [x]/[w] awaiting corrected review evidence; it becomes [p] only when substantive rework is required. [u] remains reserved for a genuine human decision as the sole next action.

The review evidence is committed first. A separate path-isolated bookkeeping commit adds Acceptance: ✓ and references the real review commit. The acceptance commit must preserve unrelated work and use compare-and-swap or equivalent expected-base protection. The reviewer never fabricates a self-referential hash.

Invalidation and reacceptance

Acceptance binds the Task-contract digest, substantive commit/tree, work-product manifest, validation profile/results, prerequisite acceptance set, authority epoch, accepted disposition, reviewer identity/assignment, and review timestamp. It is invalidated—not deleted—when relevant content or authority changes, including:

- normative Task, criterion, or Definition-of-Done change;
- accepted work-product bytes or semantic interface change;
- prerequisite acceptance invalidation or incompatible prerequisite change;
- changed required validation profile, environment, source input, or generated-output contract;
- a newly discovered material finding;
- relevant policy/authority epoch, supersession, rollback, or migration change.

An unrelated repository HEAD advance does not invalidate acceptance. Impact is determined from bound scopes, manifests, and semantics. Invalidation is append-only, names the triggering evidence, removes current acceptance credit, and propagates to affected dependent Tasks and Feature aggregate acceptance. Historical acceptance remains visible but is not current.

Feature aggregate acceptance and DONE.md

All current Task and Subtask acceptance records are necessary but not sufficient. A separately assigned independent privileged reviewer performs Feature aggregate review after every in-scope child has a current accepted disposition. The reviewer verifies:

1. complete, acyclic, current Task/Subtask and Feature-prerequisite closure;
2. satisfaction of the Feature goal and Feature Definition of Done;
3. consistency and integration of Task outputs and accepted baselines;
4. Feature-level end-to-end, negative, recovery, migration, and operational checks;
5. cross-Task findings, residual risks, exclusions, cancellations, and successors;
6. current required product/specialist approvals from their proper authorities;
7. one digest-bound aggregate manifest of Task acceptance records and work products;
8. one immutable Feature-acceptance review record.

Only after aggregate acceptance may a privileged reviewer authorize the path-isolated move to DONE.md. A grunt, checkbox counter, parent aggregation tool, or old closure-eligibility advisory cannot perform or imply this move.

Interim legacy enforcement and required migration

The human/agent authority rules in this document apply immediately. Existing legacy tools still encode [x]/[w] as terminal and therefore cannot be trusted to decide Feature closure or acceptance eligibility. Until machine enforcement is implemented:

- no automated or grunt-authored path may add, alter, invalidate, or remove acceptance records;
- no tool output stating that a Feature is closure-eligible is sufficient;
- privileged acceptance uses an exact reviewed candidate, separate evidence and bookkeeping commits, and manual verification of the rules above;
- any tool that cannot preserve acceptance records must fail closed rather than rewrite the Task block;
- future Feature 0037 approval/cutover must reconcile this process into lifecycle, schema, migration, queue, authority, and validation contracts.

Reserved follow-up Task 0039-05 owns the coordinated machine-enforcement and migration plan. It must extend the existing legacy editor/transaction semantics rather than create a competing writer, and must not appropriate active Tasks 0038-02 or 0038-05.

Measures and improvement

Track acceptance queue age, implementation-to-acceptance lead time, first-review acceptance rate, rejection/inconclusive causes, package-completeness defects, findings by severity/category, validation-rerun mismatches, escaped findings, invalidations, prerequisite-closure size, sampling escalation, independence waivers, Feature aggregate rejections/reopens, and reviewer load. Measures require definitions, denominators, population boundaries, data-quality checks, privacy contr

ols, and decision use. Acceptance volume or a high pass rate is not an assurance objective.

Automotive SPICE relationship boundary

This process can contribute evidence to quality assurance, verification, configuration management, problem/change management, project/risk/measurement management, work-product management, and process improvement. It is not an assessment and establishes no process capability level. A privileged agent is not automatically an organizationally independent QA function or competent assessor. Capability claims require the selected Automotive SPICE model/edition, named process and organizational scope, representative process instances, competent assessment, and evidence that practices are deployed and effective—not merely documented in this repository.